

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal

Turn a rough idea into a precise, buildable project description.

Instructions

Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:

Simple hospital appointment

2. Problem you want to solve (2–3 short sentences):

Patients find it difficult to book appointments quickly.
The system will help patients book & manage appointment easily

3. Target users (who will actually use this?):

Patients & hospital receptionist

4. Inputs (what data / actions will the program receive?):

- Patient name, age, doctor name, date
- & time of appointment

5. Outputs (what will the user see / get?):

- Appointment confirmation details and
- List of booked appointments

6. Constraints or assumptions (keep it realistic):

- Single hospital, limited doctors, data
- stored in file (no data base)

Part B – Core features vs “nice-to-have”

1. List minimum features needed for a working version (MVP):

- Book appointment
- View appointment
- Cancel appointment

2. List optional features (only if time allows):

- login to system, check doctor is available or not
- SMS/Email notification

3. Final question:

- If you had only 1 week, which ONE feature would you absolutely keep? Why?

Book appointment feature, because it is main purpose of the system

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal

Learn to split a project into logical modules and functions.

Instructions

Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):

- Module 1: User interface
- Module 2: appointment management
- Module 3: Data storage
- Module 4: Validation

2. For each module, write its main responsibility in one sentence:

- Module 1 does: take input from users & show output
- Module 2 does: handle book, view & cancell appointment
- Module 3 does: save & loads appointment data from file
- Module 4 does: check if input data is correct & valid

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: Appointment management

2. Functions planned:

- Function 1 name: book appointment()
 - Input(s): patient details, doctor name, date, time
 - Output: Confirmation message
- Function 2 name: view appointment()
 - Input(s): none or patient name
 - Output: List of appointment
- Function 3 name: cancel appointment
 - Input(s): appointment ID
 - Output: Cancellation message

3. Draw or briefly describe how data flows between these functions:

user enters details → Validation check data → book appointment() processes it → Data storage saves it → Confirmation shown to user.

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

Appointment

2. For this entity, list required fields:

- Field 1: name (type: int/str/float/...)

- Field 2: marks type: list

- Field 3: average type: float

- Field 4: grade type: string

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

```
item = {
```

```
    "field1": name
```

```
    "field2": marks
```

```
    ...
```

```
}
```

2. Option B – Class:

python

```
class Item:
```

```
    def __init__(self, field1, field2, ...):
```

```
        self.field1 = field1
```

```
        self.field2 = field2
```

```
        ...
```

3. Which option will you use, and why?

Class (better structure)

4. Write a short example of creating and using this structure:

python

Example creation and simple usage

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal

Build habit of designing tests before writing code.

Instructions

Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: assign-grade

2. In simple words, what should this function do?

In return grade based on average marks

3. List at least 5 test cases (inputs and expected outputs):

Test #	Input(s)	Expected output
1	90	A
2	75	B
3	60	C
4	45	D
5	30	F

4. Now write a small test harness:

python

def my_func(...):

TODO: implement

pass

Tests

print("Test 1:", my_func(...), "expected:", ...)

print("Test 2:", my_func(...), "expected:", ...)

5. Reflection:

- Did writing tests first change how you thought about the function? How?

writing test first helped me clearly decide grade rules

```
def assign-grade(avg):  
    if avg >= 85:  
        return "A"  
    elif avg >= 70:  
        return "B"  
    elif avg >= 55:  
        return "C"  
    elif avg >= 40:  
        return "D"  
    else:  
        return "F"
```

Activity A5 – Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal

Identify and protect weak points with validation and exception handling.

Instructions

Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Invalid marks
 - How to detect it? Not numeric
 - How to handle it politely for the user? Error
2. Failure Point 2: empty input
 - Detect: length is zero
 - Handle: Ask again
3. Failure Point 3: file not found
 - Detect: file not found error
 - Handle: print friendly message

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")
```

try:

```
with open(file_path, "r") as f:
```

```
    data = f.read()
```

```
    print("File loaded successfully.")
```

except FileNotFoundError:

```
    print("File not found. Please check the path.")
```

except PermissionError:

```
    print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

Your robust code:

try:
marks=list (map(int,input
('enter marks: ') split()))
except value errors:
print('please enter valid
number')

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal

Practice cleaning up messy, working code into clear, modular code.

Instructions

Take one working script (from your own work or given by teacher).

It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

Take student name & marks, calculate avg, assign grade and print results.

2. Identify at least three problems (long function, repetition, confusing names, etc.):

- Problem 1: everything written inside one long function
- Problem 2: Repeated code for calculating avg & printed results
- Problem 3: poor variable name like arbitrary

3. Plan refactoring steps (high level):

- Step 1: Divide into small functions
- Step 2: Rename variable properly
- Step 3: Remove duplicate code

4. After refactoring, write or paste your improved version here:

python

Refactored code:

5. Reflection:

- What became easier to understand after refactoring?

code become easier to read and maintain

Activity A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:

Example:

- `python student_tool.py add --name Alice --score 85`

2. What should the tool do in ONE sentence?

Add student & calculate average

3. Sketch CLI arguments:

Argument / Option	Meaning
<code>--name</code>	Student name
<code>--marks</code>	List of student marks

4. Implement with argparse (skeleton):

```
python
```

```
import argparse
```

```
parser = argparse.ArgumentParser(description="Describe your tool here")
```

```
parser.add_argument("--name", type=str, help="Student name")
```

```
parser.add_argument("--score", type=int, help="Student score")
```

```
args = parser.parse_args()
```

```
# Use args.name, args.score here
```

```
print("Name:", args.name)
```

```
print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

```
python students-tool.py --name Raaghu --marks  
python students-tool.py --name Ganesh --marks
```

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal

Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):

- Milestone 1: Design
- Milestone 2: Import implementation
- Milestone 3: Processing logic
- Milestone 4: Error handling
- Milestone 5: Testing

2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date
1	Project str diagram	Day-1
2	Working input program	Day-2
3	Grade calculation working	Day-3

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):

- Risk 1: Logic error in grade calculation
 - Plan B if this fails: Test with multiple sample input
- Risk 2: Invalid user input
 - Plan B if this fails: Add input validation & test-except block
- Risk 3: Time shortage
 - Plan B if this fails: Focus only on ~~max~~ min viable produced.

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

breaking the problem into small function & testing each part separate helped me.

2. One thing I now believe about myself as a problem-solver:

I can design project step by step by directly jumping into code.

3. If I start a new project tomorrow, what will I do differently from day 1?

I will 1st tribute the project plan define modules & design test cases before coding